ML-Powered Network Intrusion Detection System

Complete Project Documentation & Retrospective

Revanth Yepuri
MS Computer Science, George Mason University

Project period: June – July 2026 | Document written: July 30, 2026
Repository: github.com/yepurirevanth06/network-intrusion-detection-ml

Purpose of this document: a complete record of what was built, why each decision was made, every method used, and every problem encountered along the way — written so that I (or anyone) can pick this project up years from now and understand it fully.

1. Project Overview

This project is an end-to-end machine learning system that classifies network traffic as benign or malicious. It was built as the primary portfolio project for internship applications, chosen deliberately to sit at the intersection of two of my strengths: cybersecurity (from my AICTE internship) and machine learning (which my resume previously lacked hands-on evidence for). The goal was never just a trained model in a notebook — it was a production-shaped system: a real data pipeline, a served model behind a REST API with a web dashboard, containerization, an automated test suite, and a CI/CD pipeline that validates every change.

The finished system takes a CSV of network flow records, runs each flow through a trained LightGBM classifier (with a Random Forest as a second model), and returns a per-flow verdict of Attack or Benign with a confidence score, plus summary statistics across the whole file. Everything is reproducible from the repository: the Docker image builds the entire runtime, and GitHub Actions re-runs the full test suite and a container smoke test on every push.

Final results at a glance

Component	Outcome
LightGBM classifier	99.98% accuracy, 99.98% F1-score
Random Forest classifier	99.96% accuracy, 99.94% F1-score
Dataset	CICIDS2017 — 692,703 labeled flows, 78 features
Serving	Flask REST API (/health, /predict) + web dashboard, port 5001
Packaging	Docker container (python base + libgomp1)
Testing	8 pytest tests (4 model integrity, 4 API contract)
CI/CD	GitHub Actions: pytest suite + Docker build + health smoke test
Status at time of writing	AWS EC2 deployment planned (not yet done)

2. The Dataset: CICIDS2017

The data comes from the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick. It is important to remember that this is not traffic from any real company. CIC built a controlled testbed with two networks: a victim network configured like a realistic small organization (firewall, routers, switches, machines running Windows, Ubuntu and macOS) and a separate attack network that launched genuine attacks against it. The capture ran over five working days, July 3–7, 2017. Monday contained only benign traffic; the remaining days included real executions of brute force (FTP/SSH), DoS and DDoS, Heartbleed, web attacks, infiltration, botnet activity, and port scanning.

Benign traffic was made realistic through CIC's B-Profile system, which profiled the abstract behavior of 25 real users (browsing, email, FTP, SSH and similar protocols) and generated traffic that statistically mimics natural human usage — a realistic office without any real office's private data.

Raw packet captures (PCAP files) were converted into the tabular data I trained on using CICFlowMeter, a tool that groups packets into flows (conversations between two endpoints) and computes statistical features per flow: packet length statistics, inter-arrival times, flag counts, flow duration, bytes per second, and so on. Those computed features are the 78 numeric columns in my dataset; the 79th column is the label. My working dataset contained 692,703 rows.

If asked how this would work on live traffic: deploy a flow-metering capture (CICFlowMeter or equivalent) at a network tap or gateway, compute the same features in near-real-time, and feed them to the model — expecting some accuracy drop relative to the lab data, which is why in-environment retraining would be part of any production rollout.

3. Environment & Tooling Setup

Development happened on macOS in VS Code with a Python virtual environment (Python 3.14). Key packages: pandas, numpy, scikit-learn, LightGBM, imbalanced-learn, matplotlib, seaborn for the ML side; Flask for serving; pytest for testing; joblib for model persistence. Version control through Git and GitHub, with the Claude Code VS Code extension installed for assisted development. The repository lives at github.com/yepurirevanth06/network-intrusion-detection-ml.

Setup was not friction-free, and the problems are documented in Section 9 — notably a macOS folder naming issue, Xcode Command Line Tools installation, and GitHub personal-access-token authentication.

4. Data Exploration & Preprocessing

Exploration and preprocessing live in eda.ipynb. The dataset loaded as 692,703 rows by 79 columns. Data quality issues found and handled: 1,008 missing values and 1,586 infinite values. The infinities are a known artifact of CICIDS2017 — several features are ratios (for example bytes-per-second), and when a flow's duration is zero the division produces infinity. These rows/values were cleaned before training.

Features were scaled with scikit-learn's StandardScaler, and the fitted scaler was saved to models/scaler.pkl so that the API applies the exact same transformation at prediction time as was used in training. An honest technical note for future me: tree-based models (Random Forest, LightGBM) do not strictly require feature scaling — the scaler is part of the pipeline for consistency and to keep the door open for scale-sensitive models later. Knowing this distinction matters in interviews.

Visualizations produced during this phase included a confusion matrix and a feature-importance chart (saved as model_results.png in the models directory). Before any interview, re-check that chart and memorize the top three to five most important features.

5. Model Training

Two classifiers were trained for binary classification (Attack vs Benign):

Model	Accuracy	F1-Score	Notes
LightGBM	99.98%	99.98%	Primary model served by the API
Random Forest	99.96%	99.94%	Baseline / comparison model

Why tree ensembles? They handle tabular data with heterogeneous features extremely well, train quickly on ~700K rows, require far less tuning than neural networks, and expose feature importances — valuable in security contexts where analysts need to understand why traffic was flagged. The difference between the two: Random Forest is bagging (many independent trees trained in parallel, predictions averaged), while LightGBM is gradient boosting (trees built sequentially, each correcting the errors of the ensemble so far). Trained models were persisted with joblib to models/lightgbm_model.pkl and models/random_forest_model.pkl.

An important honest framing of the 99.98% number: CICIDS2017 is clean, lab-generated data where attacks have distinctive statistical signatures, so very high scores are common in the literature. On real-world traffic I would expect meaningful degradation. This is also why F1 is reported alongside accuracy — with class imbalance, accuracy alone can look excellent while missing attacks. Being upfront about this in interviews builds credibility; defending 99.98% as production-realistic would do the opposite.

A planned-vs-actual note: the original plan used XGBoost, and early drafts of the resume said XGBoost. XGBoost turned out to be incompatible with Python 3.14 at the time, so the project switched to LightGBM — which ended up performing slightly better anyway. Details in Section 9.

6. Flask REST API & Dashboard

The serving layer lives in app/app.py with the frontend in app/templates. Three routes:

```
GET  /         -> serves the dashboard (dark-themed UI, CSV upload)
GET  /health    -> {"status": "ok"}  (used by tests and the CI smoke test)
POST /predict   -> accepts multipart form upload  request.files['file']  (a CSV)
                   validates presence of the file (400-style error if missing)
                   scales features with the saved scaler, predicts per flow
                   returns JSON: { summary: {total_flows, benign_count,
                                             attack_count}, results: [...] }
                   each result: row number, prediction ('Attack'/'Benign'),
                   confidence = round(max(predict_proba) * 100, 2)
                   errors are caught and returned as {'error': ...} with HTTP 500
```

The app runs on host 0.0.0.0, port 5001. It was originally on Flask's default port 5000, but macOS reserves 5000 for the AirPlay Receiver, causing a conflict — the fix was moving to 5001 (Section 9). The dashboard lets a user upload a CSV, see a table of per-flow verdicts with confidence, and read the summary counts at the top.

Example API call

```
curl http://localhost:5001/health
curl -X POST http://localhost:5001/predict -F "file=@sample_test.csv"
```

7. Docker Containerization

The Dockerfile packages the full runtime: Python base image, pip install of requirements.txt, copying the app and models, exposing port 5001, and launching Flask. One non-obvious requirement discovered the hard way: LightGBM needs the system library libgomp1 (GNU OpenMP), which is present on a Mac dev machine but absent in slim Linux images. The container crashed on model load until an apt-get install of libgomp1 was added to the Dockerfile. This is a perfect concrete example of why Docker exists: it forced an implicit system dependency to become explicit and portable.

```
docker build -t nids-app .
docker run -p 5001:5001 nids-app   # then open http://localhost:5001
```

8. Testing & the CI/CD Pipeline

The 8-test pytest suite

tests/test_model.py verifies model integrity: the saved LightGBM model loads from disk; it knows its trained feature count (n_features_in_); it returns one prediction for one correctly-shaped input row; and the saved scaler loads and transforms input to the right shape. tests/test_api.py verifies the API contract using Flask's test client (no running server needed): /health returns 200; the dashboard route / returns 200; /predict rejects a request with no file (400/422 rather than crashing); and /predict with the real sample_test.csv returns 200 with both 'summary' and 'results' keys in the JSON.

Tests are run locally with: venv/bin/python -m pytest tests/ -v — using the venv's interpreter directly by path, for reasons explained in Section 9 (the system Python kept hijacking the plain python3 command).

The GitHub Actions workflow (.github/workflows/ci-cd.yml)

Triggered on every push and pull request to main. Two jobs run in sequence:

```
Job 1  Run Tests        ubuntu-latest, Python 3.11 (pinned deliberately)
       checkout -> setup-python (pip cache) -> pip install -r requirements.txt
       + pytest -> pytest tests/ -v

Job 2  Build Docker Image   (needs: test  -- only runs if tests pass)
       docker build -t nids-app:$GITHUB_SHA .
       docker run -d -p 5001:5001 ... ; sleep 10
       curl --fail http://localhost:5001/health  (smoke test; dumps container
       logs and fails the job if the health check does not answer)
       docker stop
```

Python 3.11 was chosen for CI rather than matching the local 3.14 because it is the battle-tested version on GitHub runners and the code uses nothing 3.14-specific. A deployment job (SSH to EC2, pull, docker compose up) is planned to be appended once the AWS instance exists. First successful full run: July 30, 2026 — Run Tests in 54s, Build Docker Image in 1m 14s.

9. Every Difficulty Faced, and How Each Was Solved

This section is the heart of the retrospective. These problems taught more than the smooth parts, and several of them are excellent interview stories. In roughly chronological order:

macOS folder naming (colons vs slashes)

Early in setup, a folder name containing special characters behaved differently than expected because macOS treats colons and slashes in file names in a legacy-swapped way. Resolved by renaming to plain alphanumeric folder names. Lesson: keep project paths boring.

Xcode Command Line Tools

Git on a fresh Mac requires Apple's Command Line Tools; the first git command triggered the install prompt. Installed via xcode-select. Lesson: expect one-time toolchain setup on any new machine.

GitHub authentication with personal access tokens

GitHub no longer accepts account passwords for HTTPS pushes; a personal access token (PAT) is required. During early setup, tokens were pasted into chat windows — a security mistake. All exposed tokens were later revoked and replaced. Lesson: a PAT is a password; it belongs in a password manager or the keychain, never in a chat, a file, or a URL.

XGBoost incompatible with Python 3.14

XGBoost would not install/run on Python 3.14 at the time of the project. Rather than downgrade the whole environment, the project switched to LightGBM, which serves the same gradient-boosting role and actually scored slightly higher. Lesson: bleeding-edge interpreter versions have ecosystem lag; have a substitute library in mind.

Port 5000 conflict with macOS AirPlay

Flask's default port 5000 is occupied by macOS's AirPlay Receiver, so the API could not bind. Moved the app to port 5001 everywhere (Flask, Dockerfile, tests, CI smoke test). Lesson: 'address already in use' on a Mac's port 5000 is almost always AirPlay.

libgomp1 missing inside Docker

The container built fine but crashed loading the LightGBM model: the slim Linux base image lacks libgomp (OpenMP), which LightGBM's native code requires. Fixed with apt-get install libgomp1 in the Dockerfile. Lesson: Python wheels can carry native dependencies; the container must provide them.

Test code kept saving into __init__.py (folder-vs-file mixup)

While creating the test suite, tests/test_model.py was accidentally created as a folder (VS Code 'New Folder' instead of 'New File'), so pasted code kept landing in tests/__init__.py, and pytest reported 'collected 0 items' (it ignores __init__.py). Diagnosed when 'rm' complained 'is a directory'. Fixed from the terminal: rm -rf the folder, mv __init__.py to test_model.py, touch a fresh empty __init__.py.

Lesson: in VS Code's explorer, a '>' arrow next to a name means folder; pytest only collects files matching test_*.py.

python3 pointing at system Python instead of the venv

pytest failed with ModuleNotFoundError: joblib even though the venv had it — the prompt showed (venv) but python3 resolved to /Library/Frameworks/.../3.14 system Python. Root cause: the venv's activation was not controlling the python3 on PATH in that shell. Fix that always works: call the interpreter by path — venv/bin/python -m pytest. Lesson: when imports mysteriously fail, print which interpreter is running; the path tells the truth.

Push rejected: PAT lacked 'workflow' scope

Pushing .github/workflows/ci-cd.yml was refused with 'refusing to allow a Personal Access Token to create or update workflow ... without workflow scope'. GitHub requires a PAT to have the workflow scope to modify CI files. Generated a new token with repo + workflow scopes. Lesson: read the rejection text literally — it named the missing scope exactly.

Old token kept being used silently (no username prompt)

Even after creating the new token, pushes failed identically and git never asked for credentials. Keychain was checked and was empty; the smoking gun was git remote -v: the old token was embedded directly in the remote URL (https://TOKEN@github.com/...), a leftover from early setup, so git never consulted any credential store. Fixed with git remote set-url origin to the clean URL; the next push prompted for username/password and the new token was cached in the macOS keychain. Lesson: when git does not prompt, the credential is coming from somewhere — check, in order, the remote URL, the keychain/credential helper, and the editor's own GitHub sign-in.

CI run #1 failed: sample_test.csv missing on the runner

First pipeline run: 7 of 8 tests passed, but the API test failed with FileNotFoundError for sample_test.csv. The file existed locally but had never been pushed — only .github/ and tests/ had been added, and a plain git add of the CSV initially did nothing (likely a .gitignore rule for CSVs, common in ML repos to keep datasets out). Fixed with git add -f sample_test.csv, then commit and push; run #2 went fully green. Lesson: CI runs only what is in the repository — 'works on my machine' often means 'a file on my machine is not in git'; verify with git status that a file is truly staged.

10. Key Lessons (the distilled version)

Reproducibility is the product: the model was perhaps thirty percent of the work; the pipeline that proves the system works on any machine was the rest. Environments lie: always know which interpreter is actually running and what files are actually in the repository. Error messages are usually literal: the workflow-scope rejection, the 'is a directory' complaint, and the FileNotFoundError each named the exact problem. Security hygiene is a habit, not an event: tokens got exposed, then revoked, then embedded in a URL, then cleaned — the end state (one scoped token, stored only in the keychain) is the pattern to start with next time. And honest metrics beat impressive metrics:

acknowledging that 99.98% is a lab-dataset number is more credible than claiming it as production performance.

11. Remaining Work at Time of Writing

AWS EC2 deployment is the one unfinished phase: launch a free-tier instance, install Docker, run the container, open the port, then append a deploy job to the CI workflow (SSH-based: git pull, docker compose up) with EC2_HOST / EC2_USER / EC2_SSH_KEY stored as GitHub Actions secrets. After deployment: add the 'deployed on AWS EC2' line back into the resume and README, and post the follow-up LinkedIn update with the live URL. Future ideas beyond that: multi-class attack-type classification, evaluation on a second dataset (CSE-CIC-IDS2018) to test generalization, and live traffic capture integration.

12. Quick Reference (for future me)

```
Repo         github.com/yepurirevanth06/network-intrusion-detection-ml
Run locally  source venv/bin/activate && python app/app.py   (port 5001)
Run tests    venv/bin/python -m pytest tests/ -v
Docker       docker build -t nids-app . && docker run -p 5001:5001 nids-app
CI file      .github/workflows/ci-cd.yml   (push to main triggers it)
Models       models/lightgbm_model.pkl, random_forest_model.pkl, scaler.pkl
Notebook     eda.ipynb  (EDA, preprocessing, training, charts)
Sample data  sample_test.csv  (also used by the API test and demos)
```

Structure: app/ (app.py + templates), models/, tests/ (test_model.py, test_api.py, __init__.py), .github/workflows/ci-cd.yml, eda.ipynb, Dockerfile, requirements.txt, README.md, sample_test.csv.

— End of document. If you are reading this years later: the debugging section is the part worth re-reading before any interview about this project.